Basilisk 101: Computational Fluid Dynamics with Basilisk C

Computational Fluid Dynamics with Basilisk: From Poiseuille Flow to Rayleigh-Bénard Convection

Vatsal Sanjay

Web version

TL;DR

This assignment explores fundamental fluid dynamics problems using Basilisk, progressing from simple Poiseuille flow to complex Rayleigh-Bénard convection. We study both Newtonian and non-Newtonian (Bingham) fluids, examine lid-driven cavity flow with flow visualization, and conclude with thermal convection simulation. The hands-on exercises with incrementally complex implementations help develop practical CFD skills while exploring rich physical phenomena, all through Basilisk's efficient simulation framework.

Introduction

In this working class, we'll explore advanced fluid dynamics simulations using the Basilisk framework. Building on our previous work with heat conduction (1st-workingAssignment), we'll now tackle more complex fluid flow problems that incorporate viscosity, non-Newtonian behavior, and thermal effects.

Learning Objectives

- Implement and analyze Poiseuille flow simulations for Newtonian and non-Newtonian fluids
- Understand the lid-driven cavity benchmark problem and its implementation
- Extend simulations with flow visualization techniques
- Explore thermal convection through the Rayleigh-Bénard configuration
- Develop skills in modifying Basilisk code for different physical scenarios

Throughout this tutorial, we'll follow a logical progression from simple to complex simulations:

1. Poiseuille flow with Newtonian fluids
2. Poiseuille flow with Bingham/non-Newtonian fluids
3. Lid-driven cavity with Newtonian fluids
4. Lid-driven cavity with Bingham fluids (exercise)
5. Flow visualization in lid-driven cavity
6. Rayleigh-Bénard convection (exercise)

Part 1: Poiseuille Flow Simulations

1.1 Newtonian Poiseuille Flow (2-Poiseuille-flow-Newtonian.c)

Let's begin with a fundamental fluid mechanics problem: Poiseuille flow with a Newtonian fluid.

Problem Statement

Solve for the steady-state velocity profile in a planar Poiseuille flow with a constant pressure gradient $-\frac{dp}{dx} = 1$ and constant viscosity μ_0. For a Newtonian fluid, the analytical solution is a parabolic profile: $u(y) = \frac{1}{2\mu_0} y(h - y)$, where $h = 1.0$ is the channel height.

The code uses Basilisk's Navier-Stokes solver to find the steady-state solution:

```
#include "navier-stokes/centered.h"

// Global parameters
char file_name[80];
double mu_0 = 1.0;        // Constant viscosity (Newtonian)
int max_iter = 1e4;       // Maximum iterations
#define DT_MAX (1e-3)     // Maximum timestep

// Initialization event
event init(t = 0) {
  // Set constant Newtonian viscosity
  const scalar mu_const[] = mu_0;
  mu = mu_const;

  // Pressure gradient (-dp/dx = 1)
  const face vector mdpdx[] = {1.0, 0.0};
  a = mdpdx;

  // Initialize velocity field at rest
  foreach() {
    u.x[] = 0;
    u.y[] = 0;
    un[] = 0;
  }
}
```

Key Implementation Details

- The domain is a unit square with periodic left/right boundaries
- No-slip boundary at the bottom ($u = 0$)
- Slip boundary at the top (zero-gradient)
- Constant horizontal pressure gradient of 1.0
- Viscosity is set to a constant value (Newtonian fluid)
- Convergence is monitored by tracking changes in the velocity field

At steady state, the simulation produces a parabolic velocity profile that can be compared with the analytical solution:

```
event end(t = end) {
  FILE *fp = fopen("profile.dat", "w");

  // Sample the velocity profile along the x-midpoint
  foreach_col(0) {
    // Calculate exact solution: u(y) = (1/2μ) * y * (h-y)
    double y = y;
    double u_exact = (1.0/(2.0*mu_0)) * y * (L0-y);
    double error = fabs((u.x[] - u_exact)/u_exact);
```

```
    fprintf(fp, "%g %g %g %g\n", y, u.x[], u_exact, error);
  }
}
```

1.2 Non-Newtonian Poiseuille Flow (2-Poiseuille-flow-Bingham.c)

Next, we'll extend our simulation to handle generalized Newtonian fluids, including Bingham plastics and power-law fluids.

Problem Statement

Solve for the steady-state velocity profile in a planar Poiseuille flow with a constant pressure gradient for a generalized Newtonian fluid that follows the Herschel-Bulkley model: $\tau = \tau_y + K(\dot{\gamma})^n$, where:

- τ_y is the yield stress
- K is the consistency index
- n is the power-law exponent
- $\dot{\gamma}$ is the strain rate

The generalized Newtonian model can represent:

- Newtonian fluid: $\tau_y = 0, n = 1$
- Power-law fluid: $\tau_y = 0, n \neq 1$
- Bingham plastic: $\tau_y > 0, n = 1$
- Herschel-Bulkley: $\tau_y > 0, n \neq 1$

The most important addition in this code is the calculation of the viscosity at each face based on the strain rate tensor:

```
event properties(i++) {
  foreach_face() {
    // Calculate deformation tensor components at face centers
    double D11 = (u.x[] - u.x[-1,0]);
    double D22 = ((u.y[0,1] - u.y[0,-1]) + (u.y[-1,1] - u.y[-1,-1])) / 4.0;
    double D12 = 0.5 * (((u.x[0,1] - u.x[0,-1]) +
              (u.x[-1,1] - u.x[-1,-1])) / 4.0 + (u.y[] - u.y[-1,0]));

    // Calculate second invariant
    double D2 = sqrt(sq(D11) + sq(D22) + 2.0 * sq(D12)) / Delta;

    // Calculate effective viscosity
    double mu_temp;
    if (D2 > 0.0) {
      double temp = tau_y / (sqrt(2.0) * D2) +
                    mu_0 * exp((n - 1.0) * log(D2 / sqrt(2.0)));
      mu_temp = min(temp, mu_max);
    } else {
      if (tau_y > 0.0 || n < 1.0) {
        mu_temp = mu_max;
      } else {
        mu_temp = (n == 1.0 ? mu_0 : 0.0);
      }
    }
```

```
    // Apply viscosity at face
    muv.x[] = fm.x[] * mu_temp;
  }
}
```

Mathematical Background For generalized Newtonian fluids, the stress tensor is related to the deformation tensor by:

$$\boldsymbol{\tau} = 2\mu_{eq}\mathcal{D} \tag{1}$$

where the equivalent viscosity μ_{eq} for a Herschel-Bulkley fluid is:

$$\mu_{eq} = \frac{\tau_y}{2|\mathcal{D}|} + K|\mathcal{D}|^{n-1} \tag{2}$$

To prevent numerical instability when $|\mathcal{D}| \approx 0$, we use regularization:

1. Compute viscosity as normal where strain rate is non-zero
2. Cap the viscosity at μ_{max} where needed
3. Handle special cases for different values of τ_y and n

This code allows us to study how different fluid models affect the velocity profile, from the parabolic profile of Newtonian fluids to the plug flow region characteristic of Bingham plastics.

Part 2: Lid-Driven Cavity Simulations

2.1 Newtonian Lid-Driven Cavity (2-LidDrivenCavity-Newtonian.c)

The lid-driven cavity is a classic benchmark problem in computational fluid dynamics.

Problem Statement

Solve for the steady-state flow in a square cavity with a moving top lid. The fluid is Newtonian with a constant viscosity, and all walls have no-slip boundary conditions except for the top wall, which moves at a constant velocity $u = 1$.

The simulation setup is straightforward:

```
#include "navier-stokes/centered.h"

// Constants
#define LEVEL   8       // Grid refinement level
#define MAXDT   (1e-4)  // Maximum timestep
#define ERROR   (1e-6)  // Convergence error threshold

// Boundary conditions
// Top moving wall (lid)
u.t[top] = dirichlet(1);
// Other no-slip boundaries
u.t[bottom] = dirichlet(0);
u.t[left]   = dirichlet(0);
u.t[right]  = dirichlet(0);
```

Lid-Driven Cavity Characteristics The lid-driven cavity flow exhibits several key features:

- Primary vortex in the center of the cavity
- Secondary corner vortices at low Reynolds numbers
- Benchmark for testing numerical schemes
- Reynolds number defined as $\mathrm{Re} = \rho UL/\mu = 1/\mu$ (with ρ=1, U=1, L=1)

At low Reynolds numbers (high viscosity), the flow reaches a steady state with well-defined vortex structures. As Re increases, the flow becomes increasingly complex.

2.2 Lid-Driven Cavity with Flow Visualization (2-LidDrivenCavity-Newtonian-dieInjection.c)

To better visualize the flow patterns, we can add a passive tracer (dye) to the simulation:

```
#include "navier-stokes/centered.h"
#include "die-injection.h"

// Die injection parameters
event init (t = 0) {
  // Set constant viscosity for Newtonian fluid
  mu = muv;

  // Initialize velocity field
  foreach() {
    u.x[] = 0;
    u.y[] = 0;
    un[] = 0;
  }
}

int main() {
  // Initialize grid and parameters
  init_grid(1<<LEVEL);
  L0 = 1.0;
  origin(-0.5, -0.5);
  DT = MAXDT;
  TOLERANCE = 1e-5;
  CFL = 0.25;

  // Die injection parameters
  tInjection = 0.05;        // Inject the die after flow is established
  xInjection = 0.00;        // X position (center of cavity)
  yInjection = 0.40;        // Y position (center of cavity)

  // Run simulation
  run();
}
```

The `die-injection.h` header defines a scalar field that is transported with the flow:

```
// Define the scalar field for the die
scalar T[];
scalar * tracers = {T};

// Initialize the die tracer to zero everywhere
```

```
event init (t = 0) {
  foreach()
    T[] = 0.0;
}

// Inject the die at the specified time
event inject_die (t = tInjection) {
  foreach() {
    double dist = sqrt(sq(x - xInjection) + sq(y - yInjection));
    if (dist <= dieRadius)
      T[] = 1.0;
  }
}
```

This tracer provides a clear visualization of the flow structures and their evolution over time, revealing the complex patterns that develop in the lid-driven cavity.

Part 3: Exercise Development

3.1 Modifying 2-LidDrivenCavity-Bingham.c for exercise

For the first exercise, I recommend modifying the lid-driven cavity simulation for Bingham fluids. Here are some suggestions for redacting parts of the code:

Exercise: Implement Bingham Plastic Model in Lid-Driven Cavity

In this exercise, you'll modify the lid-driven cavity code to simulate a Bingham plastic fluid. Complete the missing parts of the code to:

1. Calculate the deformation tensor components at face centers
2. Compute the second invariant of the deformation tensor
3. Implement the regularized Bingham viscosity model
4. Apply the viscosity at cell faces

You should run simulations with different yield stress values to observe how the flow structure changes.

Code: 2-LidDrivenCavity-Bingham_QUES.c

```
event properties(i++) {
  foreach_face() {
    // TODO: Calculate deformation tensor components
    double D11 = /* Your code here */;
    double D22 = /* Your code here */;
    double D12 = /* Your code here */;

    // TODO: Calculate second invariant (D2)
    double D2 = /* Your code here */;

    double mu_local;
    if (D2 > 0.) {
      // TODO: Implement regularized Bingham model
      double temp = /* Your code here */;
      mu_local = /* Your code here */;
```

```
    } else {
      // Handle zero strain rate case
      mu_local = (tauy > 0.0) ? mumax : MU_0;
    }

    // Apply viscosity
    muv.x[] = fm.x[] * (mu_local);
  }
}
```

1. Observe and explain the formation of unyielded regions (plug flow)
2. Compare streamline patterns for different yield stress values
3. Investigate convergence behavior as yield stress increases
4. Plot velocity profiles across horizontal or vertical sections

Solution Hints:

- Deformation tensor components must be calculated from velocity gradients
- Second invariant: $D_2 = \sqrt{D_{ij}D_{ij}}$
- Bingham model: $\mu = \tau_y/(2D_2) + \mu_0$, with regularization to prevent division by zero
- Remember to use the appropriate spatial averaging for face-centered calculations

3.2 Modifying 2-Rayleigh-Benard.c for Exercise

Exercise: Implement Rayleigh-Bénard Convection

In this exercise, you'll implement a simulation of Rayleigh-Bénard convection using the Boussinesq approximation. Complete the missing parts of the code to:

1. Set up the domain and boundary conditions
2. Initialize the temperature field with a small perturbation
3. Implement the buoyancy force in the acceleration event
4. Set up temperature diffusion
5. Calculate and monitor the Nusselt number

You'll explore how the Rayleigh number affects convection patterns.

- 2-Rayleigh-Benard.c
- convection-Boussinesq_QUES.h

Additional challenges:

1. Calculating and plotting the Rayleigh number vs. number of convection cells
2. Investigating the effect of aspect ratio on convection patterns
3. Measuring the critical Rayleigh number where convection begins
4. Implementing different thermal boundary conditions

Solution Hints:

- The Boussinesq approximation adds a buoyancy term to the y-momentum equation
- Temperature should be initialized with a linear profile plus a small random perturbation
- The acceleration term should be: $a_y = g\beta(T - T_0)$
- Use the diffusion() function from diffusion.h for temperature evolution
- The Nusselt number can be calculated from the vertical heat flux at the boundaries

Part 4: Implementation Details and Tips

4.1 Common Basilisk Patterns

Throughout these simulations, you'll notice some common patterns in Basilisk code:

Common Basilisk Patterns

1. Grid initialization:

```
init_grid(1<<LEVEL);  // Powers of 2 for efficiency
```

2. Boundary conditions:

```
u.t[top] = dirichlet(1);     // Tangential velocity
u.n[bottom] = dirichlet(0);  // Normal velocity
```

3. Events for different stages:

```
event init(t = 0) { ... }                 // Initialization
event properties(i++) { ... }            // Property updates
event integration/marching(i++) { ... }  // Time stepping
event logfile(i += 100) { ... }          // Logging
```

4. Grid traversal with foreach():

```
foreach() {
  // Operation on each cell
}
foreach_face() {
  // Operation on each face
}
```

5. Convergence monitoring:

```
double du = change(u.x, un);
if (du < ERROR) return 1; /* stop */
```

4.2 Tips for Successful Simulations

Here are some tips to help you succeed with these simulations:

Simulation Tips

- Timestep selection: For explicit schemes, ensure the timestep satisfies the relevant stability criterion (e.g., CFL condition)
- Convergence monitoring: Always check convergence by tracking changes in key variables
- Grid resolution: Start with a coarse grid for quick tests, then refine for accuracy
- Boundary conditions: Double-check that boundary conditions are correctly implemented
- Regularization parameters: For non-Newtonian fluids, choose appropriate regularization parameters to balance accuracy and stability
- Initial conditions: Use appropriate initial conditions to avoid unnecessary transients
- Output frequency: Balance output frequency with computational resources and storage needs

Conclusion

Further Exploration

Once you've completed the basic exercises, consider these extensions:

1. How does the convergence behavior change with increasing yield stress in Bingham fluids?
2. Can you identify the critical Rayleigh number where convection cells first appear?
3. How would you modify the lid-driven cavity to include temperature effects (thermally-driven cavity)?
4. What happens if you implement an oscillating lid velocity instead of constant velocity?
5. Can you extend the Poiseuille flow simulation to include time-dependent pressure gradients?

Metadata

Author:: Vatsal Sanjay Date published:: March 11, 2025 Date modified:: March 11, 2025 at 08:48 CET

Back to main website

Home, Team, Research, Github